

CSC317 Computer Graphics Tutorial 5

October 2, 2025

Assignment 5: Meshes

- Due Date: October 24 @ 11:59PM
 - Office Hour TBA
- A4 Office Hour on October 10 4-6PM

Task 1: write_obj.cpp

- [Guide to .obj file format](#)
- For every row of the matrix, print the row to one text line of the file, begins the line with what the row represents (e.g. “v”, “vn”, “f”)
- Index starts at 1

```
bool write_obj(  
    const std::string & filename,  
    const Eigen::MatrixXd & V,  
    const Eigen::MatrixXi & F,  
    const Eigen::MatrixXd & UV,  
    const Eigen::MatrixXi & UF,  
    const Eigen::MatrixXd & NV,  
    const Eigen::MatrixXi & NF)  
{
```

Task 1: write_obj.cpp

- V: vertex – v x y z
- F: face -- f v1[/vt1][/vn1]
v2[/vt2][/vn2] v3[/vt3][/vn3] ...
 - List vertices in **counterclockwise** order
 - UF: vt – index into UV mapping
 - NF: vn – index into vertex normals
- UV: UV mapping -- vt u v
- NV: vertex normal -- vn x y z

```
bool write_obj(  
    const std::string & filename,  
    const Eigen::MatrixX<double> & V,  
    const Eigen::MatrixXi & F,  
    const Eigen::MatrixX<double> & UV,  
    const Eigen::MatrixXi & UF,  
    const Eigen::MatrixX<double> & NV,  
    const Eigen::MatrixXi & NF)  
{
```

Task 2: cube.cpp

- Construct the quad mesh of a cube
- Populate the matrices V, F, etc.. with the vertices of a cube
 - Hint: in implementation it is just a long list of V, VN, UV, F...

```
// you can fill in a matrix like this :)
```

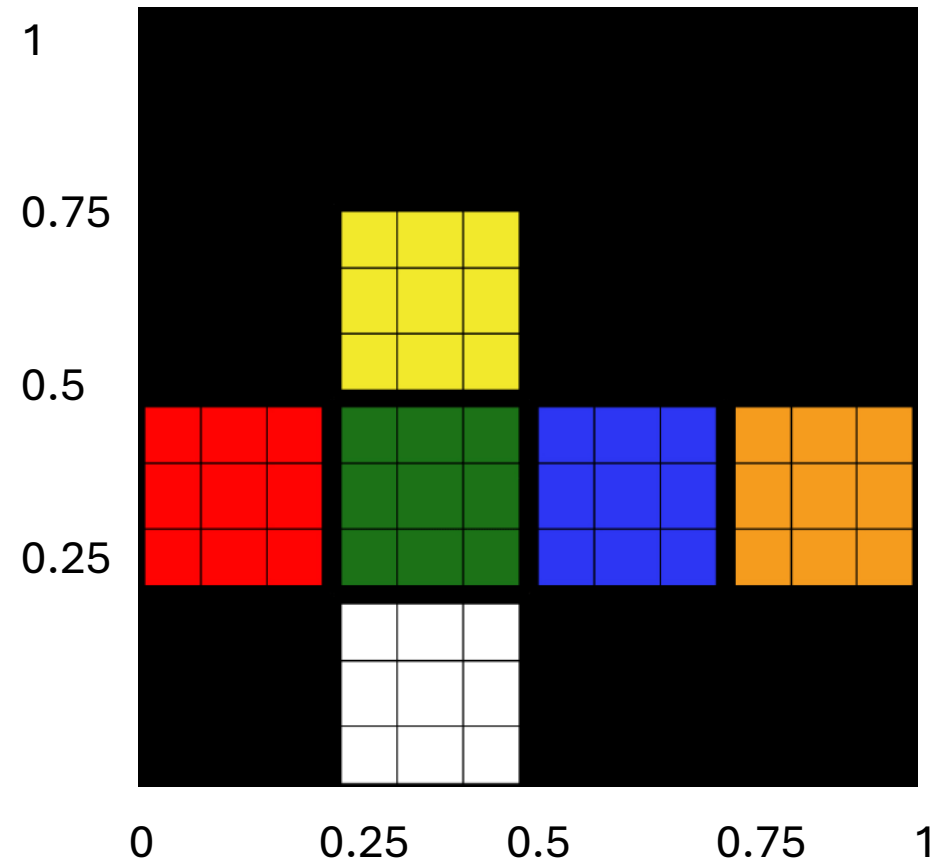
```
V.resize(8,3);
```

```
V << 0,0,0,  
      1,0,0,  
      1,1,0,  
      0,1,0,  
      0,0,1,  
      1,0,1,  
      1,1,1,  
      0,1,1;
```

```
// so you don't have to do V.row(0) = Eigen::Vector3d(0,0,0);
```

Task 2: cube.cpp

- UV maps a 3D vertex to a 2D vertex on a flattened cube
- UV (14, 2)

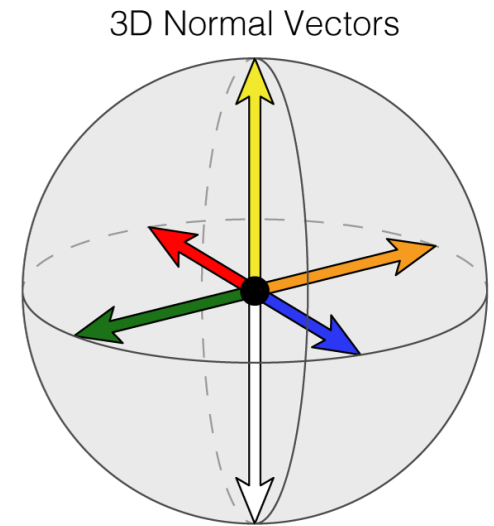
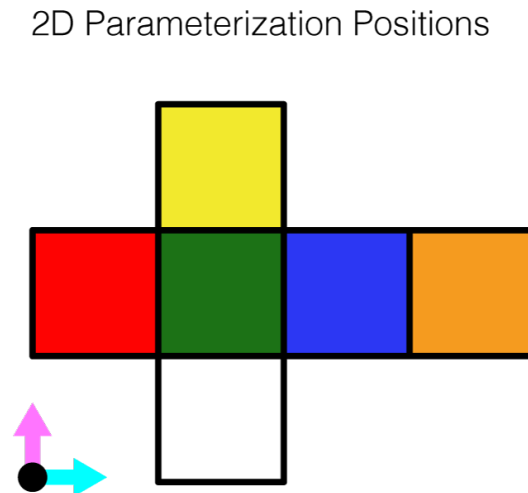
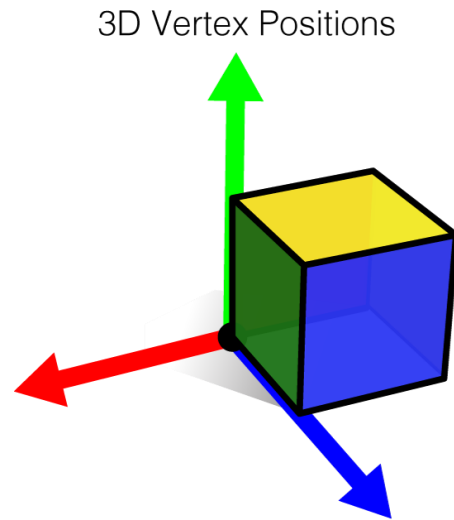


Task 2: cube.cpp

Example:

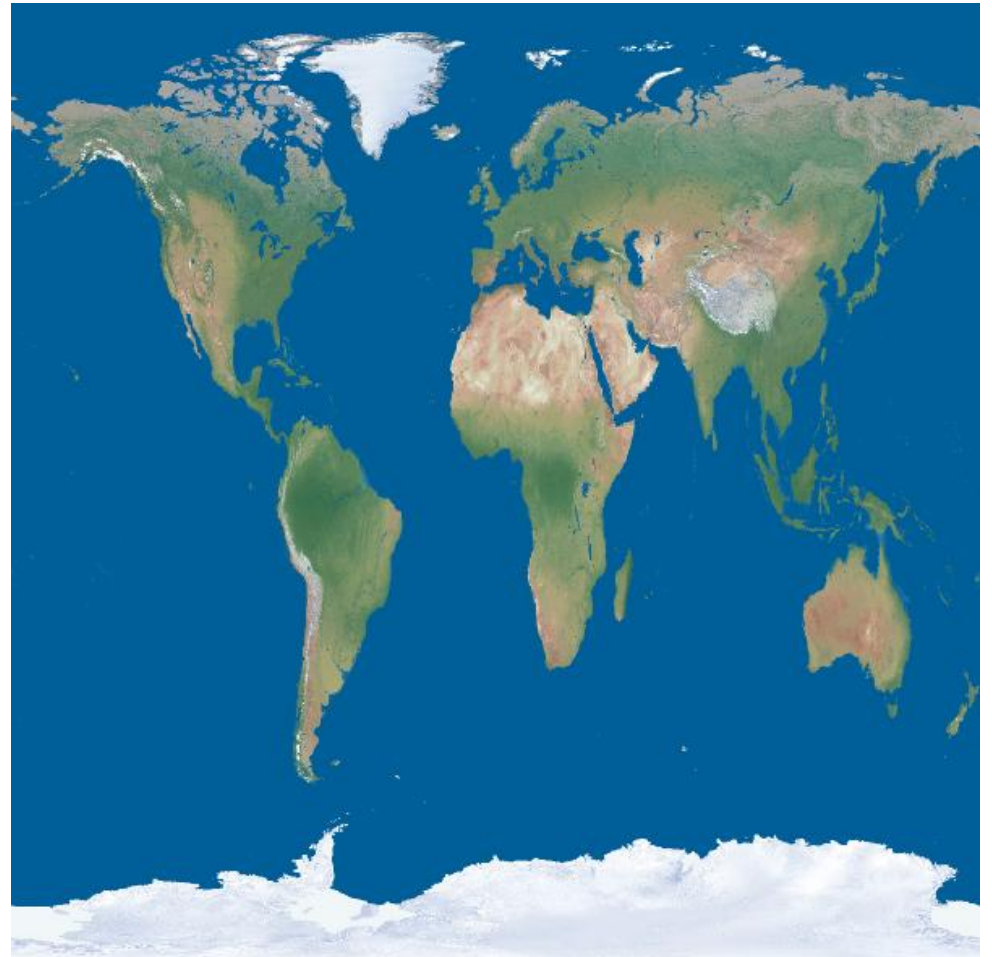
- $V(0): (0, 0, 1)$
- $UV(0): (0.25, 0)$
- $NV(0): (0, 0, 1)$
- $F(0): (0, 1, 2, 3)$
- $UF(0): (4, 5, 10, 9)$
- $NF(0): (0, 0, 0, 0)$

Yours might be in a different order (make sure vertices are counterclockwise)



Task 3: sphere.cpp

- Construct the quad mesh of a sphere with `num_faces_u` x `num_faces_v` faces → think of longitude and latitude of a globe
- Think of the UV texture of the sphere as a world map



Task 3: sphere.cpp

- Map from 2D coordinates (UV) to 3D coordinates (V) using spherical coordinates

$$(u, v) \rightarrow (r, \theta, \phi)$$

- Normal of a unit sphere is the same as its position in spherical coordinates

Task 4: triangle_area_normal.cpp

- Input: Vertices of a triangle
- Output: Normal of a triangle, where the length of the normal is the area of the triangle (not normalized normal)

n: normalized normal, A: area of the triangle

$$(p_1 - p_0) \times (p_2 - p_0) = 2A \cdot \mathbf{n}$$

Task 5: per_face_normals.cpp

- Compute per-face normal for a triangle mesh
 - Loop over all the faces of the triangle and use triangle_area_normal

Task 6: vertex_triangle_adjacency.cpp

- Compute a vertex-triangle adjacency list VF
 - F (num_face, 3) stores the vertices per face of the mesh. VF (num_vertices, _) stores the list of incident faces for each vertex.

```
void vertex_triangle_adjacency(  
    const Eigen::MatrixXi & F,  
    const int num_vertices,  
    std::vector<std::vector<int> > & VF)
```

Task 7: per_vertex_normal.cpp

- Compute per-vertex normal for a triangle mesh
- Use vertex_triangle_adjacency.cpp to find the adjacent faces of a vertex, then average the normal of those faces

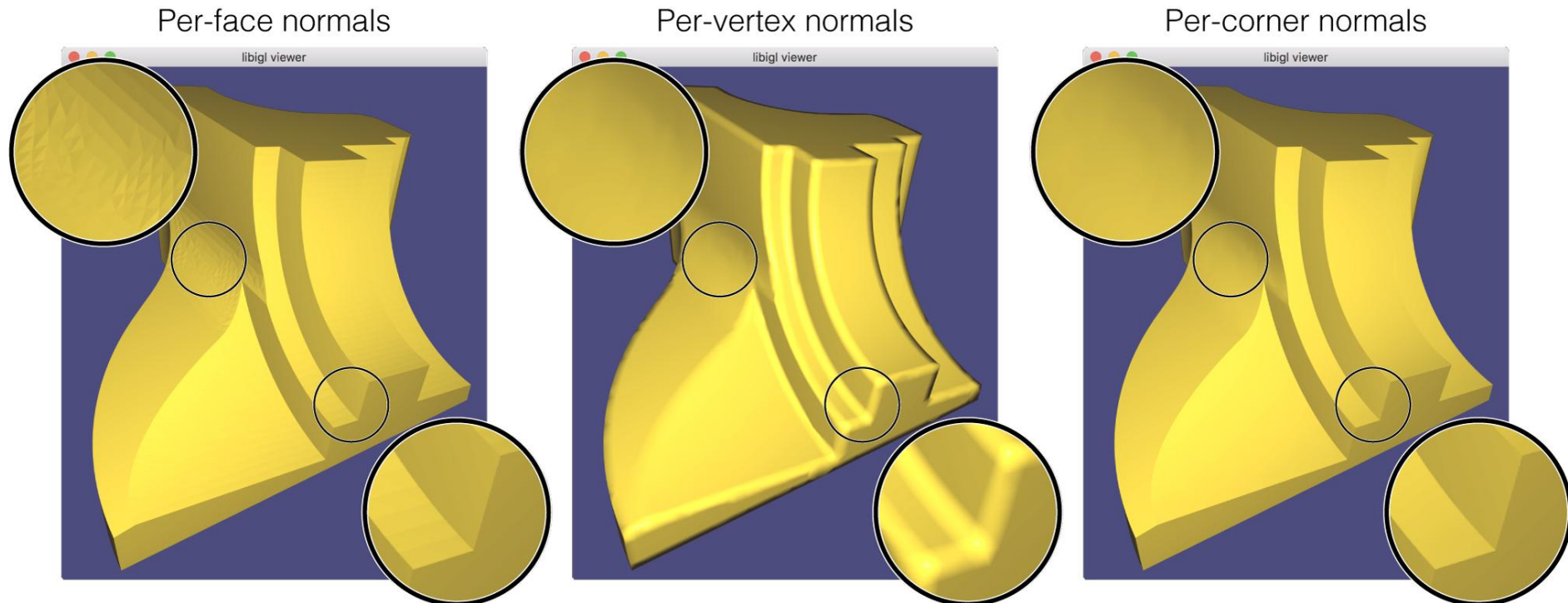
$$\mathbf{n}_v = \frac{\sum_{f \in Nb(v)} A_f \cdot \mathbf{n}_f}{\left\| \sum_{f \in Nb(v)} A_f \cdot \mathbf{n}_f \right\|}$$

where A_f is the area of the face (triangle) f , $\mathbf{n}_f$ is the normalized normal of the face (triangle), $Nb(v)$ is the set of faces that contains vertex v .

- Loop over neighbouring faces of the vertex, sum up the area-weighted normals of the faces (triangle_area_normal.cpp), then normalize the sum.

Task 8: per_corner_normals.cpp

- Per face normal is defined over singular triangles so global smoothness is not guaranteed
- Per vertex normals smooth out sharp edges too much



Task 8: per_corner_normals.cpp

- Define a normal for every triangle corner (as opposed to each mesh vertex). If the angle between two neighbouring faces is greater than a threshold, do not average the normal

$$\mathbf{n}_{f,c} = \frac{\sum_{g \in Nb(c) | \mathbf{n}_f \cdot \mathbf{n}_g > \epsilon} A_g \cdot \mathbf{n}_g}{\left\| \sum_{g \in Nb(c) | \mathbf{n}_f \cdot \mathbf{n}_g > \epsilon} A_g \cdot \mathbf{n}_g \right\|}$$

where A_g is the area of the face (triangle) g , $\mathbf{n}_g$ is the normalized normal of the face (triangle), $Nb(c)$ is the set of faces that contains triangle f corner c .

- Only consider pairs of faces where the dot product is greater than ϵ

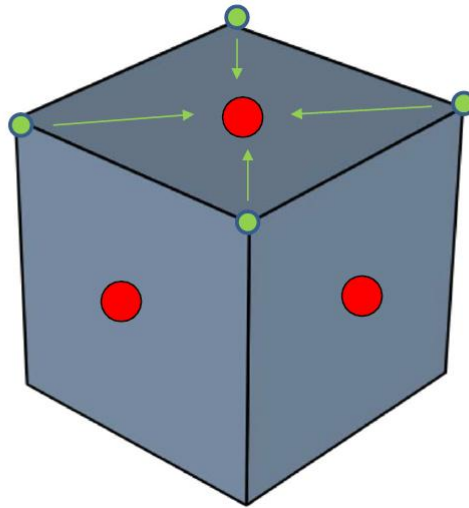
Task 8: per_corner_normals.cpp

- Corner threshold is the minimum angle between two per face normals
 - Convert dot product to angle before threshold checking
- Think of how to find the neighbouring faces of a face
- Hint: can use 3 nested for loops

```
void per_corner_normals(  
    const Eigen::MatrixXd & V,  
    const Eigen::MatrixXi & F,  
    const double corner_threshold,  
    Eigen::MatrixXd & N)
```

Task 9: catmull_clark.cpp

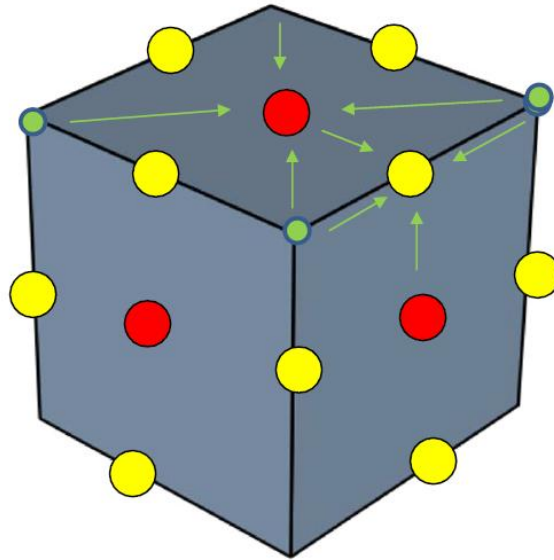
- Step 1: Set the face point for each facet to be the average of its vertices



Task 9: catmull_clark.cpp

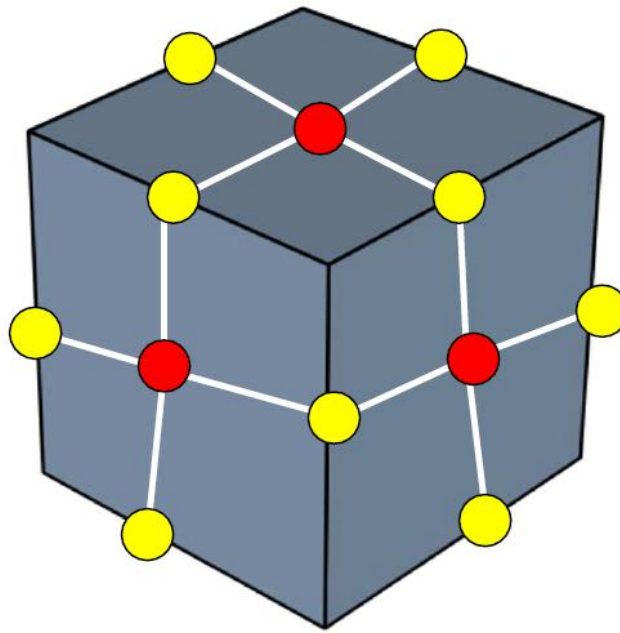
- Step 2: Add edge points – average of the two neighbouring face points and edge end points

*new edge points are not necessarily on the original edge



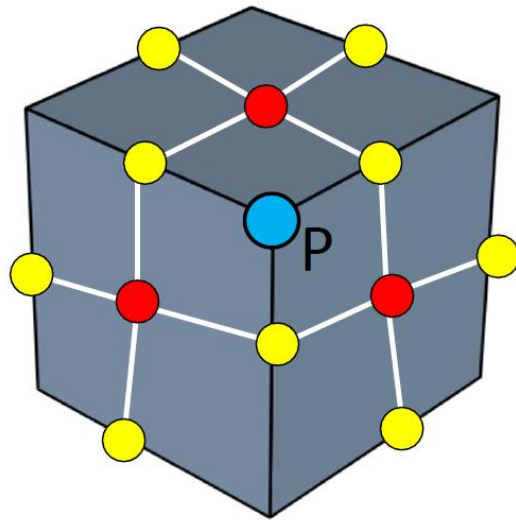
Task 9: catmull_clark.cpp

- Step 3: Add edges between face points and edge points



Task 9: catmull_clark.cpp

- Step 4: Move each original vertex according to new position given by:



$$\frac{F + 2R + (n - 3)P}{n}$$

F : Average of all n created face points adjacent to P

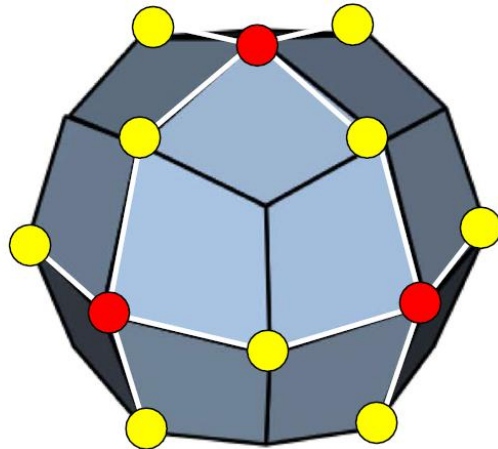
R : Average of all original edge midpoints touching P

n = number of edges connected to P

Task 9: catmull_clark.cpp

- Step 4: Move each original vertex according to new position given by:

$$\frac{F + 2R + (n - 3)P}{n}$$



F : Average of all n created
face points adjacent to P

R : Average of all original edge
midpoints touching P

Task 9: catmull_clark.cpp

- Step 5: Connect up original points to make facets

